PAPERBOY - TECHNICAL DOCUMENTATION

County of Ventura GSA Forms Platform

Last Updated: March 9, 2026

Rails Version: 8.0.2

Ruby Module: FormsApp

TABLE OF CONTENTS

1. APPLICATION OVERVIEW 2. TECHNOLOGY STACK & DEPENDENCIES 3. DATABASE ARCHITECTURE 4. AUTHENTICATION & SESSION MANAGEMENT 5. AUTHORIZATION & ACCESS CONTROL (ACL) 6. MODELS - CORE AUTHENTICATION & HIERARCHY 7. MODELS - ORGANIZATIONAL HIERARCHY (LOOKUP TABLES) 8. MODELS - FORM TEMPLATE SYSTEM (DYNAMIC FORM BUILDER) 9. MODELS - LEGACY FORM SUBMISSIONS 10. MODELS - GENERATED FORM SUBMISSIONS 11. MODELS - SUPPORT & UTILITY 12. MODEL CONCERNS (MIXINS) 13. SERVICES 14. CONTROLLERS - INFRASTRUCTURE 15. CONTROLLERS - FORM SUBMISSIONS 16. CONTROLLERS - ADMIN & AUTHORIZATION 17. CONTROLLERS - REPORTS & DASHBOARDS 18. CONTROLLERS - DATA MANAGEMENT & LOOKUPS 19. CONTROLLERS - BILLING & INVOICES 20. CONTROLLERS - API & EXTERNAL INTEGRATIONS 21. CONTROLLER CONCERNS 22. ROUTES 23. VIEWS & LAYOUTS 24. JAVASCRIPT & STIMULUS CONTROLLERS 25. STYLESHEETS & DESIGN SYSTEM 26. HELPERS 27. MAILERS 28. BACKGROUND JOBS 29. RAKE TASKS 30. CONFIGURATION FILES 31. TESTING 32. DEPLOYMENT & INFRASTRUCTURE 33. KEY ARCHITECTURAL PATTERNS 34. DATA FLOW DIAGRAMS

1. APPLICATION OVERVIEW

Paperboy is a Rails 8 internal web application built for the County of Ventura General Services Agency (GSA). It serves as a centralized forms platform that handles:

- Dynamic form creation and management (no-code form builder)
- Form submission workflows with multi-step approval routing
- Task inbox for supervisors/managers to approve, deny, or reassign
- Employee submission tracking with status timelines
- Report generation (CSV/PDF) with scheduling
- Power BI dashboard embedding
- Billing tools and invoice generation via MSSQL stored procedures
- Access control at both group and organizational hierarchy levels
- Help desk ticketing system
- Lookup table management for reference data

The app is used by County of Ventura employees to submit various operational forms (parking permits, probation transfer requests, critical incident reports, OSHA forms, leave of absence, workplace violence reports, etc.) and by managers/administrators to process those submissions.

Primary URL: gsa-forms (internal) Module Name: FormsApp

2. TECHNOLOGY STACK & DEPENDENCIES

BACKEND:
- Ruby on Rails 8.0.2 (load_defaults 8.0)
- Ruby (system Ruby)
- Puma web server (>= 5.0)

DATABASE:
- Microsoft SQL Server (GSABSS) via tiny_tds + activerecord-sqlserver-adapter
- Primary connection: All application tables
- Billing connection: Separate pool to same DB for stored procedures
- PostgreSQL: Commented out, reserved for future migration
- Redis: For Sidekiq job queue (redis://localhost:6379/0)

FRONTEND:
- Hotwire Turbo (turbo-rails) - SPA-like page acceleration

- Hotwire Stimulus (stimulus-rails) - Modest JavaScript framework
- Import Maps (importmap-rails) - No bundler/webpack needed
- SCSS via sass-rails (>= 6) + sass-embedded (~> 1.94)
- SortableJS (1.15.2) - Drag-and-drop reordering
- No propshaft (commented out)

AUTHENTICATION:
- OmniAuth with Entra ID (Azure AD) via omniauth-entra-id
- omniauth-rails_csrf_protection for CSRF safety
- Duo Web for two-factor authentication (duo_web gem)

BACKGROUND JOBS:
- Sidekiq (primary job processor)
- sidekiq-cron (scheduled/recurring jobs)
- Redis as job queue backend

PDF GENERATION:
- Prawn (core PDF rendering)
- prawn-table (table layouts in PDFs)
- prawn-templates (PDF template overlays)

REPORTS:
- csv gem (CSV generation)
- rubyzip (ZIP file bundling for PDF report packages)

DEPLOYMENT:
- Kamal (Docker-based deployment, require: false)
- Thruster (HTTP asset caching/compression, require: false)
- Bootsnap (boot time reduction via caching)

DEVELOPMENT/TEST:
- debug gem (Ruby debugger)
- brakeman (security static analysis)
- rubocop-rails-omakase (Ruby style linting)
- web-console (in-browser debugging)
- capybara + selenium-webdriver (system testing)
- dotenv-rails (environment variable management, all environments)

CACHING/QUEUES (development/test only):
- solid_cache
- solid_queue
- solid_cable (ActionCable adapter for production)

IMPORTMAP PINS:
- application (main entry)
- @hotwired/turbo-rails -> turbo.min.js
- @hotwired/stimulus -> stimulus.min.js
- @hotwired/stimulus-loading -> stimulus-loading.js
- All controllers from app/javascript/controllers/
- form_navigation -> form_navigation.js
- sortablejs -> https://esm.sh/sortablejs@1.15.2

==
3. DATABASE ARCHITECTURE
==

3.1 CONNECTION CONFIGURATION (config/database.yml) ----------

PRIMARY CONNECTION (default):
  Adapter: sqlserver
  Host: ENV['GSABSS_HOST']
  Database: ENV['GSABSS_DATABASE']
  Username: ENV['GSABSS_USERNAME']
  Password: ENV['GSABSS_PASSWORD']
  Pool: ENV['RAILS_MAX_THREADS'] (default 5)
  Encoding: utf8

BILLING CONNECTION (billing):
  Same server and database as primary
  Separate connection pool for billing stored procedures
  Accessed via BillingBase abstract model
  Used for: TC60 exports, Invoice queries, Monthly billing, Backups

NOTE: PostgreSQL configuration is commented out in database.yml and Gemfile. The pg gem is also commented out. This is reserved for a future migration.

3.2 TABLE CATEGORIES ----------

CORE TABLES (PascalCase columns, Rails-managed):
  - Employees (PK: EmployeeID)
  - Groups (PK: GroupID)
  - Employee_Groups (Composite PK: EmployeeID, GroupID)
  - Group_Permissions (Composite PK: GroupID, Permission_Type, Permission_Key)

ORGANIZATIONAL HIERARCHY (snake_case, read-only lookup tables):
  - agencies (PK: agency_id, 3 char)
  - divisions (Composite PK: agency_id, division_id)
  - departments (Composite PK: agency_id, division_id, department_id)
  - units (Composite PK: agency_id, division_id, department_id, unit_id)
  - activities (Composite PK: agency_id, activity_id)
  - functions (Composite PK: agency_id, function_id)
  - funds (PK: fund_id, 4 char)
  - department_funds (Composite PK: agency_id, fund_id)
  - major_programs (Composite PK: agency_id, major_program_id)
  - programs (Composite PK: agency_id, program_id, major_program_id)
  - phases (Composite PK: agency_id, phase_id)
  - tasks (Composite PK: agency_id, task_id)
  - revenue_sources (PK: revenue_id, smallint)
  - objects (PK: object_id, smallint)
  - sub_objects (Composite PK: agency_id, object_id, sub_object_id)
  - sub_units (Composite PK: agency_id, unit_id, subunit_id)

FORM SYSTEM TABLES (snake_case, Rails-managed):
  - form_templates
  - form_fields
  - form_template_statuses
  - form_template_routing_steps
  - org_permissions

FORM SUBMISSION TABLES (snake_case, Rails-managed):
  - parking_lot_submissions + parking_lot_vehicles
  - probation_transfer_requests
  - critical_information_reportings
  - carpool_forms
  - gym_locker_forms
  - social_media_forms
  - brown_mail_forms
  - leave_of_absence_forms
  - work_schedule_or_location_update_forms
  - workplace_violence_forms
  - notice_of_change_forms
  - osha301_forms
  - rm75_forms
  - creative_job_requests
  - (Additional dynamically-generated form tables)

SUPPORT TABLES (snake_case, Rails-managed):
  - authorization_managers
  - authorized_approvers
  - saved_searches
  - help_tickets
  - status_changes (polymorphic)

- task_reassignments (polymorphic)
- scheduled_reports

ACTIVE STORAGE TABLES:
- active_storage_attachments
- active_storage_blobs
- active_storage_variant_records

BILLING TABLES (read-only via BillingBase):
- TC60, TC60_Adjustments, TC60_Services, TC60_Types
- Invoices
- Manual_Transactions, PlanVsActual
- BdmRates, BdmRateTypes, BusinessUnit
- CustomerAccount, CustomerAccountWithType
- GL218Detail, FiscalMonths, FiscalYears
- AimUsers, Employees_Old

3.3 MSSQL COLUMN NAMING CONVENTION ----------

CRITICAL: Only 4 models use PascalCase MSSQL column names with alias_attribute mappings. All other tables (35+) use standard snake_case columns.

The 4 PascalCase tables are:
1. Employee (table: Employees)
2. Group (table: Groups)
3. EmployeeGroup (table: Employee_Groups)
4. GroupPermission (table: Group_Permissions)

IMPORTANT: alias_attribute works with where(), order(), select(), find_by(), exists?(), and group() â but pluck(:id) does NOT translate to the PK column. You must use pluck(:GroupID) explicitly for PascalCase tables.

==========
4. AUTHENTICATION & SESSION MANAGEMENT
==========

4.1 AUTHENTICATION FLOW ----------

PRIMARY: OAuth via Microsoft Entra ID (Azure AD)
Route: GET /auth/callback -> SessionsController#create_oauth
Flow:
1. User clicks login -> redirected to Entra ID
2. Entra ID authenticates -> redirects back with OmniAuth callback
3. SessionsController#create_oauth extracts email from callback
4. Finds Employee record by email (EE_Email column)
5. Populates session hash with employee data
6. Redirects to root_path

DEVELOPMENT AND TEST: Legacy login by employee id
Route: POST /login -> SessionsController#create_legacy
Other environments return 403 and set no session.
Flow:
1. Caller posts an employee id
2. Finds Employee by id. Impersonation is separate.

LOGOUT:
Route: DELETE /logout -> SessionsController#destroy
Clears entire session and redirects to root

4.2 SESSION DATA STRUCTURE ----------

```
session[:user] = {
  "employee_id"   => Integer,    # Employee.EmployeeID
  "email"         => String,     # Employee.EE_Email
  "first_name"    => String,     # Employee.First_Name
  "last_name"     => String,     # Employee.Last_Name
```

"phone" => String, # Employee.Work_Phone
"supervisor_id" => Integer, # Employee.Supervisor_ID
"department" => String, # Employee.Department
"agency" => String, # Employee.Agency
"unit" => String # Employee.Unit }

session[:user_id] = Integer # Quick-access copy of employee_id

4.3 CURRENT ATTRIBUTES ----------

File: app/models/current.rb Type: ActiveSupport::CurrentAttributes

Current.user stores the session[:user] hash for the duration of each request, set by ApplicationController#set_current_user. This makes user data accessible in models and other non-controller contexts.

4.4 SESSION USER MODEL ----------

File: app/models/session_user.rb Type: Plain Ruby class (not ActiveRecord)

Purpose: Wraps Employee for authentication and group-checking operations.

Class Methods:
authenticate(emp_id_or_email) -> SessionUser instance or nil

Instance Methods:
employee -> Employee ActiveRecord object
in_group?(name) -> Boolean (checks group membership)
in_any_group?(*names) -> Boolean (checks any group membership)

4.5 ADMIN IMPERSONATION ----------

File: app/controllers/admin/impersonations_controller.rb Routes: /admin/impersonations (new, create, destroy)

Flow:
1. Admin navigates to impersonation page
2. Selects employee from list
3. System stores session[:real_admin_id] = current employee ID
4. Session replaced with impersonated employee's data
5. Yellow "Emulating" banner shows in header
6. Admin can destroy impersonation to restore their own session

Guard: require_super_admin (checks session for admin privileges)

==========
5. AUTHORIZATION & ACCESS CONTROL (ACL)
==========

5.1 GROUP-BASED PERMISSIONS ----------

Groups are stored in the Groups table (PascalCase: GroupID, Group_Name). Employees are assigned to groups via Employee_Groups join table. Group permissions are stored in Group_Permissions table.

KEY SYSTEM GROUPS (checked by name, stored downcased):
- "system_admins" -> Full admin access (system_admin? helper)
- "auth_console_admin" -> Authorization console admin access
- "auth_console_approvers" -> Authorization console user access
- "ticket_admin" -> Help ticket administration
- "status managers" -> Submission status management

5.2 PERMISSION TYPES ----------

Two permission types exist in Group_Permissions:

DROPDOWN PERMISSIONS (permission_type: 'dropdown'):
Control visibility of navigation/sidebar items:

- inbox -> Inbox queue access
- submissions -> Submissions tracker access
- reports -> Report generation access
- dashboards -> Power BI dashboard access
- manage_forms -> Form template builder access
- emulate -> Admin impersonation access
- acl -> ACL management access
- auth_console -> Authorization console access
- lookup_tables -> Lookup table management access

FORM PERMISSIONS (permission_type: 'form'):
Control access to specific restricted form templates.
Permission_Key = FormTemplate.id

5.3 ORGANIZATION-LEVEL PERMISSIONS ----------

Table: org_permissions Model: OrgPermission

Permissions can also be scoped to organizational hierarchy levels:
- Agency level (agency_id only)
- Division level (agency_id + division_id)
- Department level (agency_id + division_id + department_id)
- Unit level (agency_id + division_id + department_id + unit_id)

Permission types: 'dropdown' or 'form' These cascade: a permission at agency level applies to all divisions, departments, and units within that agency.

5.4 PERMISSION RESOLUTION (ApplicationController) ----------

current_user_group_names:
- Memoized Set of downcased group names
- Built via EmployeeGroup.joins(:group).where(EmployeeID: user_id)
- Used for system_admin?, auth_console_admin?, etc.

current_user_group_ids:
- Memoized Array of GroupIDs
- Used for form permission lookups

current_user_org_chain:
- Returns hash: { agency_id, division_id, department_id, unit_id }
- Built from session[:user] data

load_user_permissions(permission_type):
- Cascades org-level permissions (agency -> division -> department -> unit)
- Merges with group-level permissions
- Returns Set of permission keys

current_user_dropdown_permissions:
- Calls load_user_permissions('dropdown')
- Controls sidebar navigation visibility

current_user_form_permission_ids:
- Calls load_user_permissions('form')
- Controls which restricted forms the user can access

5.5 FORM ACCESS CONTROL ----------

FormTemplate has two access levels:
- "public" -> All authenticated employees can access
- "restricted" -> Access controlled by org_scope OR acl_group_id

Restricted forms check access via FormTemplate#accessible_by?(user_org_chain, user_group_ids):
- If org_scoped?: checks user's org chain matches org_scope_type/org_scope_id
- If acl_group_id set: checks user's group IDs include the ACL group

5.6 CONTROLLER-LEVEL GUARDS ----------

before_action :require_system_admin
  -> Redirects to root_path unless system_admin?
  -> Used by: FormTemplatesController, AclController, ReportsController,
     BillingToolsController, ScheduledReportsController

before_action :require_auth_console
  -> Redirects unless auth_console_admin? or auth_console_user?
  -> Used by: AuthorizationConsoleController

Custom per-controller checks:
  -> ticket_admin? in HelpTicketsController/HelpController
  -> require_authentication in DashboardsController
  -> require_login in TaskReassignmentsController

==
6. MODELS - CORE AUTHENTICATION & HIERARCHY
==

6.1 EMPLOYEE ----------

File: app/models/employee.rb Table: Employees Primary Key: EmployeeID (integer) Inherits: Application-Record

ALIAS ATTRIBUTES (PascalCase -> snake_case):
  employee_id   -> EmployeeID
  first_name   -> First_Name
  last_name   -> Last_Name
  email   -> EE_Email
  work_phone   -> Work_Phone
  supervisor_id   -> Supervisor_ID
  supervisor_first_name -> Supervisor_First_Name
  supervisor_last_name -> Supervisor_Last_Name
  job_title   -> Job_Title
  job_code   -> Job_Code
  job_class   -> Job_Class
  pay_status   -> Pay_Status
  union_code   -> Union_Code
  employee_type   -> Type
  agency   -> Agency
  department   -> Department
  unit   -> Unit
  position   -> Position

ASSOCIATIONS:
  has_many :employee_groups (FK: EmployeeID)
  has_many :groups (through: employee_groups)

INSTANCE METHODS:
  in_group?(name)   -> Check if employee belongs to named group
  in_any_group?(*names)   -> Check if employee belongs to any of the named groups

6.2 GROUP ----------

File: app/models/group.rb Table: Groups Primary Key: GroupID (integer) Inherits: ApplicationRecord

ALIAS ATTRIBUTES:
  group_name   -> Group_Name
  description -> Description
  created_at   -> Created_At

COLUMNS:
  GroupID   (integer, auto-increment)
  Group_Name   (nvarchar 100, NOT NULL)
  Description (nvarchar 500, nullable)
  Created_At   (datetime, default: GETDATE())

ASSOCIATIONS:
  has_many :employee_groups (FK: GroupID, dependent: :destroy)
  has_many :employees (through: :employee_groups)
  has_many :group_permissions (FK: GroupID, dependent: :destroy)

6.3 EMPLOYEE GROUP ----------

File: app/models/employee_group.rb Table: Employee_Groups Primary Key: Composite (EmployeeID, GroupID) Inherits: ApplicationRecord

ALIAS ATTRIBUTES:
  employee_id -> EmployeeID
  group_id   -> GroupID
  assigned_at -> Assigned_At
  assigned_by -> Assigned_By

COLUMNS:
  EmployeeID   (integer, NOT NULL)
  GroupID   (integer, NOT NULL)
  Assigned_At (datetime, default: GETDATE())
  Assigned_By (integer, nullable)

ASSOCIATIONS:
  belongs_to :employee (FK: EmployeeID, PK: EmployeeID)
  belongs_to :group (FK: GroupID)

6.4 GROUP PERMISSION ----------

File: app/models/group_permission.rb Table: Group_Permissions Primary Key: Composite (GroupID, Permission_Type, Permission_Key) Inherits: ApplicationRecord

ALIAS ATTRIBUTES:
  group_id   -> GroupID
  permission_type -> Permission_Type
  permission_key   -> Permission_Key

COLUMNS:
  GroupID   (integer, NOT NULL)
  Permission_Type (nvarchar 50, NOT NULL)
  Permission_Key   (nvarchar 255, NOT NULL)
  Created_At   (datetime, default: GETDATE())

ASSOCIATIONS:
  belongs_to :group (FK: GroupID)

==========
7. MODELS - ORGANIZATIONAL HIERARCHY (LOOKUP TABLES)
==========

The organizational hierarchy represents the County of Ventura's structure: Agency -> Division -> Department -> Unit

Additional reference tables exist for budgeting and program classification.

7.1 AGENCY ---------- Table: agencies | PK: agency_id (string, 3 char) Columns: agency_id, name (100), long_name (100), short_name (50)

7.2 DIVISION ---------- Table: divisions | PK: Composite (agency_id, division_id) Columns: agency_id, division_id (4 char), long_name (100), short_name (50)

7.3 DEPARTMENT ---------- Table: departments | PK: Composite (agency_id, division_id, department_id) Columns: agency_id, division_id, department_id (4 char), long_name (100),
short_name (50)

7.4 UNIT --------- Table: units | PK: Composite (agency_id, division_id, department_id, unit_id) Columns:

agency_id, division_id, department_id, unit_id (4 char),
long_name (100), short_name (50) Associations:
belongs_to :agency (FK: agency_id, optional)
belongs_to :division (FK: division_id, optional)
belongs_to :department (FK: department_id, optional)

7.5 ADDITIONAL HIERARCHY TABLES ---------- Activity: PK: (agency_id, activity_id) | long_name, short_name Function: PK: (agency_id, function_id) | long_name, short_name Fund: PK: fund_id (4 char) | long_name, short_name, fund_class,
fund_category, fund_type, fund_group, cafr_type DepartmentFund: PK: (agency_id, fund_id) MajorProgram: PK: (agency_id, major_program_id) | long_name, short_name Program: PK: (agency_id, program_id, major_program_id) | long_name,
short_name Phase: PK: (agency_id, phase_id) | long_name, short_name Task: PK: (agency_id, task_id) | long_name, short_name RevenueSource: PK: revenue_id (smallint) | long_name, short_name Object: PK: object_id (smallint) | long_name, short_name SubObject: PK: (agency_id, object_id, sub_object_id) SubUnit: PK: (agency_id, unit_id, subunit_id) | short_name

==========
8. MODELS - FORM TEMPLATE SYSTEM (DYNAMIC FORM BUILDER)
==========

The Form Template system allows administrators to create forms without writing code. It generates database tables, models, controllers, and views dynamically.

8.1 FORM TEMPLATE ----------

File: app/models/form_template.rb Table: form_templates Primary Key: id (bigint, auto-increment)

KEY COLUMNS:
name (string, NOT NULL, unique)
class_name (string, NOT NULL, unique) - Generated from name
access_level (string, default: "public") - "public" or "restricted"
acl_group_id (integer, nullable) - FK to Groups for ACL
page_count (integer, default: 2, range: 2-30)
page_headers (JSON) - Header text per page
created_by (integer) - Employee ID of creator
submission_type (string, default: "database") - "database" or "approval"
approval_routing_to (string, nullable) - Routing strategy
approval_employee_id (integer, nullable) - Specific employee for routing
powerbi_workspace_id (string, nullable) - Power BI workspace
powerbi_report_id (string, nullable) - Power BI report
has_dashboard (boolean, default: false)
inbox_buttons (JSON array) - Enabled inbox action buttons
status_transition_mode (string, default: "automatic") - "automatic" or "manual"
tags (string) - Comma-separated searchable tags
org_scope_type (string, nullable) - "agency"/"division"/"department"/"unit"
org_scope_id (string, nullable) - ID of scoped org unit

ASSOCIATIONS:
has_many :form_fields (dependent: :destroy)
has_many :routing_steps -> FormTemplateRoutingStep (ordered by step_number,
dependent: :destroy)
has_many :statuses -> FormTemplateStatus (ordered by position,
dependent: :destroy)
accepts_nested_attributes_for :routing_steps, :statuses

VALIDATIONS:
- name: presence, uniqueness
- class_name: presence, uniqueness
- access_level: inclusion in ['public', 'restricted']
- page_count: 2 to 30
- org_scope_type: inclusion (allow nil)
- restricted forms must have scope_or_group
- submission_type: inclusion in ['database', 'approval']
- approval_routing_to: required if requires_legacy_routing?

- approval_employee_id: required if routes_to_specific_employee?
- powerbi_workspace_id + powerbi_report_id: required if has_dashboard?
- status_transition_mode: inclusion in ['automatic', 'manual']

SCOPES:
with_dashboards - where(has_dashboard: true)

KEY METHODS:
restricted? -> access_level == 'restricted'
requires_approval? -> submission_type == 'approval'
routes_to_specific_employee? -> requires_approval && routing_to == 'employee'
has_routing_steps? -> routing_steps.any? || pending steps present
has_multiple_routing_steps? -> routing_steps.count > 1
has_dashboard? -> has_dashboard == true
automatic_status_transitions? -> status_transition_mode == 'automatic'
manual_status_transitions? -> status_transition_mode == 'manual'
has_inbox_button?(type) -> Check if button_type in inbox_buttons
enabled_inbox_buttons -> Return inbox_buttons array
table_name -> class_name.underscore.pluralize
file_name -> class_name.underscore
accessible_by?(org_chain, group_ids) -> Check org scope + ACL group
acl_group_name -> Get Group_Name of acl_group_id
page_header(page_num) -> Get header text for specific page
tags_array / tags_array= -> Serialize tags as array
all_tags (class method) -> Get all unique tags across templates
org_scoped? -> org_scope_type.present? && org_scope_id.present?
org_scope_label -> Get long_name of scoped org unit

CONSTANTS:
INBOX_BUTTON_TYPES - Hash of 7 button types:
view_pdf, edit, approve, deny, reassign, take_back, status_dropdown
TRANSITION_MODES - %w[automatic manual]

CALLBACKS:
before_validation :generate_class_name (on: :create)
before_validation :clear_restrictions_if_public
validate :page_count_cannot_orphan_fields (on: :update)

8.2 FORM FIELD ----------

File: app/models/form_field.rb Table: form_fields Primary Key: id (bigint)

KEY COLUMNS:
form_template_id (bigint, FK, NOT NULL)
field_name (string, NOT NULL) - Generated from label
field_type (string, NOT NULL) - text/text_box/dropdown/date
label (string) - Display label
page_number (integer, NOT NULL, 1-30)
position (integer) - Order within page
options (JSON) - Type-specific options (rows, values)
required (boolean, default: false)
restricted_to_type (string, default: "none") - none/employee/group
restricted_to_employee_id (integer) - Specific employee restriction
restricted_to_group_id (integer) - Group restriction
conditional_field_id (integer) - FK to another FormField
conditional_values (JSON array) - Values that make this field visible

ASSOCIATIONS:
belongs_to :form_template

SCOPES:
for_page(n) - where(page_number: n).order(:position)
ordered - order(:page_number, :position)
unrestricted - where(restricted_to_type: ['none', nil])
restricted - where.not(restricted_to_type: ['none', nil])
dropdowns - where(field_type: 'dropdown')

conditional - where.not(conditional_field_id: nil)
unconditional - where(conditional_field_id: nil)

KEY METHODS:
text_field?, text_box?, dropdown?, date? - Type checks
rows -> Get 'rows' from options (default 5)
dropdown_values -> Get 'values' from options
restricted?, unrestricted? -> Restriction state
editable_by?(employee_id, user_groups) -> Check if user can fill field
required_for?(employee_id, user_groups) -> Check if required for user
conditional?, unconditional? -> Conditional state
conditional_field -> Get the field this depends on
visible_for_value?(value) -> Check if visible given a dropdown value
restriction_label -> Human-readable restriction description
conditional_label -> Human-readable conditional description

CONSTANTS:
FIELD_TYPES - %w[text text_box dropdown date]
RESTRICTION_TYPES - %w[none employee group]

CALLBACKS:
before_validation :generate_field_name (on: :create)
before_validation :set_default_restriction_type
before_save :set_default_options

8.3 FORM TEMPLATE STATUS --------------------------

File: app/models/form_template_status.rb Table: form_template_statuses Primary Key: id (bigint)

KEY COLUMNS:
form_template_id (bigint, FK, NOT NULL)
name (string, NOT NULL)
key (string, NOT NULL, unique per template)
category (string, NOT NULL) - pending/in_review/approved/denied/
cancelled/scheduled
position (integer, default: 0)
is_initial (boolean, default: false) - Initial status for new submissions
is_end (boolean, default: false) - Terminal status

ASSOCIATIONS:
belongs_to :form_template

SCOPES:
ordered - order(:position)
initial - where(is_initial: true)
end_status - where(is_end: true)
non_end - where(is_end: false)

CONSTANTS:
VALID_CATEGORIES - %w[pending in_review approved denied cancelled scheduled]
PREDEFINED_STATUSES - Array of 10 common statuses with default mappings

8.4 FORM TEMPLATE ROUTING STEP --------------------------------

File: app/models/form_template_routing_step.rb Table: form_template_routing_steps Primary Key: id (bigint)

KEY COLUMNS:
form_template_id (bigint, FK, NOT NULL)
step_number (integer, NOT NULL, unique per template)
routing_type (string, NOT NULL) - supervisor/department_head/employee
employee_id (integer) - Required if routing_type == 'employee'
form_template_status_id (bigint, FK, optional) - Status to apply at this step

ASSOCIATIONS:
belongs_to :form_template

belongs_to :form_template_status (optional)

SCOPES:
ordered - order(:step_number)

KEY METHODS:
routes_to_employee? -> routing_type == 'employee'
routing_label -> Human-readable routing description
employee_name -> Get employee's name for display
status_to_apply -> Returns associated form_template_status

CONSTANTS:
ROUTING_TYPES - %w[supervisor department_head employee]

==
9. MODELS - LEGACY FORM SUBMISSIONS
==

Legacy forms are hardcoded form types that existed before the dynamic form builder. They have dedicated models, controllers, views, and PDF generators.

9.1 PARKING LOT SUBMISSION ----------

File: app/models/parking_lot_submission.rb Table: parking_lot_submissions Concerns: PhoneNumberable, Reassignable, TrackableStatus

KEY COLUMNS:
name, phone, employee_id, email (employee info)
agency, division, department, unit (org hierarchy codes)
status (enum: submitted=0, pending_delegated_approval=1, denied=2,
approved=3, sent_to_security=4)
supervisor_id, supervisor_email
approved_by, approved_at, denied_by, denied_at, denial_reason
delegated_approver_id, delegated_approver_email
delegated_approved_by, delegated_approved_at

ASSOCIATIONS:
belongs_to :agency_record (Agency, FK: agency, PK: agency_id)
belongs_to :division_record (Division, FK: division)
belongs_to :department_record (Department, FK: department)
belongs_to :unit_record (Unit, FK: unit)
has_many :parking_lot_vehicles (dependent: :destroy)
accepts_nested_attributes_for :parking_lot_vehicles

APPROVAL WORKFLOW:
1. Employee submits -> status: submitted (0)
2a. If MB3 union -> auto-approved, sent_to_security
2b. Supervisor approves -> status: pending_delegated_approval (1)
3. Delegated approver (Sean Payne) approves -> status: approved (3)
4. System sets sent_to_security (4) and emails SecurityMailer

KEY METHODS:
*_long_name (agency/division/department/unit) -> Org unit display names
unit_display -> "unit_id - unit_long_name"
status_label -> Humanized status
current_assignee_id -> delegated_approver_id or supervisor_id
assignment_field_name -> 'delegated_approver_id' or 'supervisor_id'

SCOPES:
for_employee(employee_id)

STATUS_CATEGORIES: Maps statuses to normalized categories for badge display

9.2 PARKING LOT VEHICLE ----------

File: app/models/parking_lot_vehicle.rb Table: parking_lot_vehicles

COLUMNS:
  parking_lot_submission_id (FK), make, model, color, year,
  license_plate, parking_lot, other_parking_lot

ASSOCIATIONS:
  belongs_to :parking_lot_submission

KEY METHOD:
  display_parking_lot -> "Other: {other}" if parking_lot == "Other"

9.3 PROBATION TRANSFER REQUEST ---------------------------------

File: app/models/probation_transfer_request.rb Table: probation_transfer_requests Concerns: PhoneNumberable, Reassignable, TrackableStatus

KEY COLUMNS:
  employee_id, name, email, phone (employee info)
  agency, division, department, unit (org hierarchy)
  work_location, current_assignment_date
  desired_transfer_destination (text, semicolon-separated multi-select)
  other_transfer_destination
  status (integer: submitted=0, manager_approved=1, denied=2, sent_to_security=3)
  approved_by, approved_at, denied_by, denied_at, denial_reason
  supervisor_id, supervisor_email
  expires_at (datetime) - 1 year from creation
  canceled_at, canceled_reason
  superseded_by_id (bigint) - Links to replacement request
  approved_destination

LIFECYCLE:
  - Requests expire after 1 year (ProbationRequestsExpiryJob runs daily at 2am)
  - Creating a new request cancels existing ACTIVE requests
  - Canceled requests show superseded_by_id linking to replacement
  - Users can withdraw (cancel) their own requests

KEY METHODS:
  desired_destinations_array -> Parse semicolon/comma/pipe-separated values
  ensure_expires!   -> Set expires_at to 1 year from created_at
  cancel!(reason:)   -> Mark as canceled
  expire_if_due!   -> Mark as canceled if expires_at has passed
  current_assignee_id   -> supervisor_id (always)

SCOPES:
  for_employee(id), not_canceled, not_expired, active

STATUS_MAP: {0=>"submitted", 1=>"manager_approved", 2=>"denied", 3=>"sent_to_security"}

9.4 CRITICAL INFORMATION REPORTING -----------------------------------

File: app/models/critical_information_reporting.rb Table: critical_information_reportings Concerns: Reassignable, TrackableStatus

KEY COLUMNS:
  employee_id, name, phone, email (reporter info)
  agency, division, department, unit (org hierarchy)
  incident_type, incident_details (text), cause (text)
  staff_involved (text)
  impact_started (datetime), location
  actual_completion_date
  urgency (Low/Medium/High/Immediate)
  impact (Low/Medium/High)
  impacted_customers, next_steps
  status (enum: in_progress=0, resolved=1, scheduled=2, cancelled=3)
  assigned_manager_id

ASSOCIATIONS:
  has_one_attached :media (ActiveStorage - JPG/PNG/GIF/PDF, max 10MB)

SPECIAL BEHAVIOR:
  - before_validation :assign_manager_based_on_location (on create)
    Uses CriticalInformationLocationRouter service to auto-assign manager
  - If urgency is "Immediate", NotifyTeamsJob is triggered on create
    (sends Microsoft Teams adaptive card notification)

KEY METHODS:
  high_impact?   -> impact == 'High'
  immediate_urgency?   -> urgency == 'Immediate'
  assigned_manager   -> Employee record
  assigned_manager_name -> Display name
  current_assignee_id   -> assigned_manager_id

SCOPES:
  for_employee(id), by_status(s), by_impact(i), recent,
  high_urgency, assigned_to(manager_id)

9.5 RM75 FORM (OCCUPATIONAL INJURY/ILLNESS) ----------

File: app/models/rm75_form.rb Table: rm75_forms Concerns: TrackableStatus

STATUS: in_progress=0 (default), approved=1, denied=2, cancelled=3

KEY COLUMNS (injury/incident focused):
  report_type, bloodborne_pathogen_exposure
  supervisor_name, witness_name, witness_phone
  date_of_injury_or_illness, date_employer_notified
  date_dwc1_given, who_gave_the_dwc1
  date_last_worked, date_returned_to_work
  missed_full_work_day_, still_off_work
  specific_injury_and_body_part_affected
  location_of_incident, on_employer_premises
  department_where_event_occurred
  activity_at_time_of_incident, how_the_injury_occurred
  physician_name/address/phone, hospital_name/address/phone
  hospitalized_overnight
  investigator_name/title/phone
  nature_of_incident, cause_of_incident, root_cause
  assessment_of_future_severity_potential
  assessment_of_probability_of_recurrence
  unsafe_condition_corrected_immediately
  checklistprocedurestraining_modified
  person_responsible_for_corrective_action
  targeted_completion_date, actual_completion_date
  osha_recordable, osha_reportable, reportable_injury_codes

ASSOCIATIONS:
  has_one :osha301_form

KEY METHOD:
  create_osha301! -> Creates pre-filled Osha301Form from RM75 data using
                     OSHA301_FIELD_MAP constant

CONSTANTS:
  GARY_HOWARD_ID = '135272' (default approver for OSHA forms)
  OSHA301_FIELD_MAP - Maps RM75 fields to Osha301Form fields

9.6 OSHA 301 FORM ----------

File: app/models/osha301_form.rb Table: osha301_forms Concerns: TrackableStatus

STATUS: in_progress=0, approved=1

Detailed OSHA injury/illness reporting form with employee info, injury details, physician info, facility info, and case narrative fields.

ASSOCIATIONS:
  belongs_to :rm75_form (optional)

==
10. MODELS - GENERATED FORM SUBMISSIONS
==

These models were either generated by the Form Template Builder or created with simple form patterns. They follow a common structure.

10.1 LEAVE OF ABSENCE FORM
  Table: leave_of_absence_forms
  Concerns: TrackableStatus
  Status: in_progress=0, approved=1, denied=2, cancelled=3
  Columns: employee_id, name, phone, email, agency, division, department,
           unit, status, approver_id, deny_reason

10.2 WORK SCHEDULE OR LOCATION UPDATE FORM
  Table: work_schedule_or_location_update_forms
  Status: submitted=0, step_1_pending=1, step_1_approved=2,
          step_2_pending=3, approved=4, denied=5
  Multi-step approval workflow (2 approval steps)

10.3 WORKPLACE VIOLENCE FORM
  Table: workplace_violence_forms
  Concerns: TrackableStatus
  Status: in_progress=0, approved=1, denied=2, cancelled=3

10.4 NOTICE OF CHANGE FORM
  Table: notice_of_change_forms
  Concerns: TrackableStatus
  Status: in_progress=0, approved=1, denied=2, cancelled=3

10.5 SIMPLE SUBMIT-ONLY FORMS
  CarpoolForm   (table: carpool_forms, status enum: submitted/pending/denied/approved)
  GymLockerForm   (table: gym_locker_forms, validates: name, email)
  SocialMediaForm   (table: social_media_forms, validates: name, email)
  BrownMailForm   (table: brown_mail_forms, status enum: submitted/pending/denied/approved)
  CreativeJobRequest (table: creative_job_requests, minimal model)

==
11. MODELS - SUPPORT & UTILITY
==

11.1 STATUS CHANGE (Polymorphic)
  Table: status_changes
  Polymorphic: belongs_to :trackable (polymorphic: true)
  Columns: trackable_type, trackable_id, from_status, to_status,
           changed_by_id, changed_by_name, notes
  Scopes: chronological, reverse_chronological

11.2 TASK REASSIGNMENT (Polymorphic)
  Table: task_reassignments
  Polymorphic: belongs_to :task (polymorphic: true)
  Columns: task_type, task_id, from_employee_id, to_employee_id,
           reassigned_by_id, reason, assignment_field
  Methods: from_employee, to_employee, reassigned_by_employee

11.3 SAVED SEARCH
  Table: saved_searches
  Columns: employee_id, name, filters (JSON)
  Validations: employee_id, name presence; name unique per employee
  Scopes: for_employee(id)

11.4 HELP TICKET
  Table: help_tickets
  Status: open=0, closed=1
  Columns: subject, description, employee_id, employee_name,
        employee_email, status
  Scopes: for_employee(eid)

11.5 AUTHORIZATION MANAGER
  Table: authorization_managers
  Columns: employee_id, department_id
  Methods: employee, department (lookup records)

11.6 AUTHORIZED APPROVER
  Table: authorized_approvers
  Columns: employee_id, department_id, service_type, key_type,
        span, budget_units (JSON), locations (JSON), authorized_by
  SERVICE_TYPES: P=Parking, E=Employee ID, V=Volunteer ID,
            C=Vendor ID, K=Keys
  KEY_TYPES: 7 types of keys/access cards
  SPANS: A=All, B-E=progressively more specific
  Methods: approvers_for(department_id:, service_type:),
        approver_for_unit(department_id:, service_type:, unit_id:)

11.7 SCHEDULED REPORT
  Table: scheduled_reports
  Columns: employee_id, form_type, format (csv/pdf),
        status_filter, frequency (daily/weekly/monthly),
        time_of_day (HH:MM), day_of_week, day_of_month,
        date_range_type, enabled, last_run_at, next_run_at
  Methods: calculate_date_range, calculate_next_run, execute!,
        form_template, schedule_description

11.8 ORG PERMISSION
  Table: org_permissions
  Columns: agency_id, division_id, department_id, unit_id,
        permission_type, permission_key
  Unique Index: all columns combined

11.9 INVOICE QUERY (Plain Ruby)
  File: app/models/invoice_query.rb
  Method: fetch(fiscal_year:, agency:, account:) -> Stored proc call

11.10 TC60 EXPORT (Plain Ruby)
  File: app/models/tc60_export.rb
  Method: call(sDate:, eDate:, type:, digits:, encumbered:) -> Stored proc call

11.11 BILLING BASE (Abstract)
  File: app/models/billing_base.rb
  Configuration: connects_to database: { writing: :billing, reading: :billing }
  Purpose: All billing models extend this for separate connection pool

==
12.   MODEL   CONCERNS   (MIXINS)
==

12.1 PHONE NUMBERABLE
  File: app/models/concerns/phone_numberable.rb
  Behavior:
    before_validation :normalize_phone! -> Strip all non-digits
    validate :phone_has_ten_digits -> Ensure exactly 10 digits
  Used By: ParkingLotSubmission, ProbationTransferRequest,
        CriticalInformationReporting

12.2 TRACKABLE STATUS
  File: app/models/concerns/trackable_status.rb

Associations: has_many :status_changes (polymorphic, dependent: destroy)
Callbacks:
after_create :record_initial_status
after_update :record_status_change (if status changed)
Class Methods:
status_label_for(value) -> Human label
status_category_for(value) -> Normalized category
statuses_for_category(cat) -> All statuses in category
category_labels -> Label hash
Instance Methods:
status_timeline -> Chronological status_changes
status_category -> Current normalized category
status_category_label -> Human label of category
Requirement: Models MUST define STATUS_CATEGORIES constant mapping
statuses to categories (pending/in_review/approved/denied/
cancelled/scheduled)
Used By: ParkingLotSubmission, ProbationTransferRequest,
CriticalInformationReporting, LeaveOfAbsenceForm,
WorkplaceViolenceForm, NoticeOfChangeForm, Osha301Form,
Rm75Form, WorkScheduleOrLocationUpdateForm

12.3 REASSIGNABLE
File: app/models/concerns/reassignable.rb
Associations: has_many :task_reassignments (polymorphic, dependent: destroy)
Abstract Methods (must be implemented by model):
current_assignee_id -> Return current assignee's employee ID
assignment_field_name -> Return column name storing assignee
Instance Methods:
reassign_to!(new_assignee_id:, reassigned_by_id:, reason:)
reassignment_chain -> History of reassignments
can_take_back?(emp_id) -> Check if employee can reclaim
take_back!(emp_id) -> Reassign back to earlier assignee
assignee_history -> All employees involved with task
Used By: ParkingLotSubmission, ProbationTransferRequest,
CriticalInformationReporting

==========
13. SERVICES
==========

13.1 PDF GENERATORS ----------

All PDF generators use the Prawn gem and include the Ventura County logo from app/assets/images/Ventura_Logo.png.

ParkingLotPdfGenerator
- Generates parking permit submission PDF
- Includes employee info, org hierarchy, vehicle list
- Used by: ParkingLotSubmissionsController#pdf, SecurityMailer

ProbationTransferPdfGenerator
- Generates probation transfer request PDF
- Includes employee info, work location, destinations
- Used by: ProbationTransferRequestsController#pdf, ProbationMailer

CriticalInformationPdfGenerator
- Generates critical incident report PDF
- Handles media embedding (images/PDFs via tempfiles)
- Used by: CriticalInformationReportingsController#pdf

WorkScheduleOrLocationUpdatePdfGenerator
- Generates schedule/location update form PDF
- Includes org hierarchy lookups

LeaveOfAbsenceFormPdfGenerator
- Generates leave of absence form PDF

WorkplaceViolenceFormPdfGenerator
  - Generates workplace violence incident form PDF

Osha301FormPdfGenerator
  - Generates OSHA 301 incident form PDF

Rm75FormPdfGenerator
  - Generates RM75 occupational injury form PDF

13.2 BUSINESS LOGIC SERVICES ----------

FormGenerator (app/services/form_generator.rb)
  Purpose: Generates complete form CRUD scaffolding from FormTemplate
  Generates:
    - Model with attributes and validations
    - Database migration with proper MSSQL indexing
    - Controller with access checks
    - Views (new.html.erb) with multi-page support
    - Routes entry in config/routes.rb
    - Sidebar link in application layout
  Supports: Multi-page forms, ACL-aware access control, field types
            (text, text_box/textarea, dropdown, date)

TeamsNotifier (app/services/teams_notifier.rb)
  Purpose: Send adaptive cards to Microsoft Teams webhook
  Method: send_cir_alert(cir) -> Builds and POSTs Teams adaptive card
  Card includes: incident type, location, urgency, impact, reporter,
                 submission time, truncated details, "View in Paperboy" link
  Config: Rails.application.config.teams_webhook_url
  Timeout: 10s read, 5s open
  Error handling: Logs failures, does not raise

PowerBiService (app/services/power_bi_service.rb)
  Purpose: Generate Power BI embed tokens for government cloud
  Method: generate_embed_token(workspace_id, report_id)
  Returns: { token: string, expiration: datetime }
  Environment Variables:
    POWERBI_CLIENT_ID, POWERBI_CLIENT_SECRET, POWERBI_TENANT_ID
    POWERBI_AUTHORITY_URL (default: login.microsoftonline.com)
    POWERBI_API_URL (default: api.powerbigov.us - government cloud)
    POWERBI_SCOPE (default: analysis.usgovcloudapi.net scope)
  Future: Row-Level Security (RLS) support planned

CriticalInformationLocationRouter (app/services/critical_information_location_router.rb)
  Purpose: Auto-assign facility managers based on incident location
  Method: find_manager_for_location(location) -> employee_id or nil
  Managers (5 configured):
    Vincent Childs (121520), Colleen Cardona (126471),
    Joey Carmona (124502), Gerald Urias (137554),
    Steve Blair (120131), Nathan Paul (128975)
  Matching Strategy:
    1. Substring matching (most reliable)
    2. City + address parsing
    3. Street number matching
    4. 70% word overlap threshold for street names
  Normalizes: uppercase, punctuation removal, abbreviation expansion
              (ST->STREET, AVE->AVENUE, etc.)
  200+ hardcoded location-manager pairs

==
14. CONTROLLERS - INFRASTRUCTURE
==

14.1 APPLICATION CONTROLLER ----------

File: app/controllers/application_controller.rb Base controller for all others. No authentication before_action by default; each controller decides its own guards.

AUTHENTICATION HELPERS:
  current_user()   -> SessionUser from session[:user]
  set_current_user()   -> Sets Current.user for request lifecycle

AUTHORIZATION HELPERS:
  current_user_group_names()   -> Memoized Set of downcased group names
  current_user_group_ids()   -> Memoized Array of group IDs
  current_user_org_chain()   -> {agency_id, division_id, department_id, unit_id}
  auth_console_admin?()   -> "system_admins" or "auth_console_admin"
  auth_console_user?()   -> auth_console_admin? OR "auth_console_approvers"
  require_system_admin()   -> before_action guard -> redirect if not admin
  require_auth_console()   -> before_action guard for auth console

PERMISSION HELPERS:
  current_user_dropdown_permissions() -> 'dropdown' type permission keys
  current_user_form_permission_ids() -> 'form' type permission keys as Set
  load_user_permissions(type)   -> Cascades org + group permissions

DATA BUILDERS:
  build_prefill_data(employee_id)   -> Hash with employee/org info for forms
  inbox_count()   -> Count of pending inbox items

14.2 SESSIONS CONTROLLER --------------------------

File: app/controllers/sessions_controller.rb

Actions:
  create_oauth() -> Handles OmniAuth/Entra ID callback
    - Extracts email from OmniAuth auth hash
    - Finds Employee by email
    - Populates session hash
    - Redirects to root

  create_legacy() -> Development and test employee-id sign-in
    - Returns 403 and sets no session outside those environments
    - Finds Employee by employee_id; impersonation is separate

  failure() -> Handles OAuth failures
  destroy() -> Clears session, redirects to root

14.3 SHARED CONTROLLER ------------------------

File: app/controllers/shared_controller.rb
  form_success() -> Renders generic success page after form submission

==
15. CONTROLLERS - FORM SUBMISSIONS
==

15.1 SUBMISSIONS CONTROLLER (Unified View) --

File: app/controllers/submissions_controller.rb Includes: Filterable concern

PURPOSE: Unified view of ALL form submissions for the current user. Combines legacy forms (ParkingLot, ProbationTransfer, CriticalInfo) with dynamically-generated form submissions.

CONSTANTS:
  LEGACY_FORMS - [ParkingLotSubmission, ProbationTransferRequest,
                  CriticalInformationReporting]
  FORM_TYPE_TO_MODEL - Maps type name strings to model classes

ACTIONS:
  index()

- Sets @is_manager based on "status managers" group
- Loads legacy form submissions via load_legacy_forms()
- Loads dynamic form submissions via load_form_template_submissions()
- Applies filters via Filterable concern
- Builds status options map for JavaScript dynamic filtering
- Renders sortable table with saved search support

status_options()
- AJAX endpoint returning statuses for a selected form type
- Used by status_filters Stimulus controller

HELPERS:
load_legacy_forms() -> Queries each legacy model for current employee
load_form_template_submissions() -> Queries FormTemplate models
generate_submission_path() -> Dynamic path helper generation
build_status_item() -> Builds hash: {id, type, title, status, path, ...}

15.2 INBOX CONTROLLER (Approval Queue) ----------

File: app/controllers/inbox_controller.rb Includes: Filterable concern

PURPOSE: Shows submissions awaiting current user's approval/action.

ACTION: queue()
Loads pending items across all form types:
- ParkingLotSubmissions where supervisor_id OR delegated_approver_id matches
- ProbationTransferRequests where supervisor_id matches (not canceled)
- CriticalInformationReportings where assigned_manager_id matches
- Dynamic forms where approver_id matches current user
Applies filters and sorting via Filterable
Loads employee list for reassignment modal

HELPERS:
fetch_dynamic_form_submissions() -> Queries FormTemplates with
submission_type='approval'

15.3 PARKING LOT SUBMISSIONS CONTROLLER ----------

File: app/controllers/parking_lot_submissions_controller.rb

ACTIONS:
new() -> Form with org hierarchy and parking lot options
- Checks MB3 union code for R Lot access
- Loads agency/division/department/unit dropdown options
- Builds @prefill_data from session employee

create() -> Submission with approval routing
- Auto-approves if MB3 union (status=1, no supervisor needed)
- Routes to AuthorizedApprover if non-MB3
- Triggers NotifySecurityJob for MB3 auto-approvals

show() -> Read-only submission view
pdf() -> PDF via ParkingLotPdfGenerator (inline display)
index() -> Redirects to inbox_queue_path

approve() -> Multi-step approval workflow:
Step 1: submitted -> pending_delegated_approval
(Routes to Sean Payne as delegated approver)
Step 2: pending_delegated_approval -> approved + sent_to_security
(Sends SecurityMailer notification)

deny() -> Sets status=denied, stores denial_reason
Sends SecurityMailer.denied notification

STRONG PARAMS: name, phone, email, agency, division, department, unit,
parking_lot_vehicles_attributes[] REJECTS: status, employee_id (set server-side for security)

15.4 PROBATION TRANSFER REQUESTS CONTROLLER ----------

File: app/controllers/probation_transfer_requests_controller.rb

ACTIONS:
  new() -> Shows existing request gate if ACTIVE request within 1 year
    - Allows ?proceed=1 to bypass gate and create new
    - Builds form with work location options (18 hardcoded locations)
    - Shows multi-select transfer destinations

  create() -> Submission with lifecycle management
    - Fetches supervisor_id from employee
    - Joins multi-select destinations with semicolons
    - Cancels older ACTIVE requests (sets canceled_at, superseded_by_id)

  show() -> Responds to HTML and PDF format
  pdf() -> Via ProbationTransferPdfGenerator
  withdraw() -> Cancels request, redirects to new with ?proceed=1

  approve() -> Approval with destination selection
    - If request has multiple options, requires selected destination
    - Triggers NotifyProbationJob

  deny() -> Denial with reason (status=denied)

STRONG PARAMS: employee_id, name, phone, email, agency, division,
               department, unit, work_location, current_assignment_date,
               other_transfer_destination, desired_transfer_destination[]

15.5 CRITICAL INFORMATION REPORTINGS CONTROLLER ----------

File: app/controllers/critical_information_reportings_controller.rb

ACTIONS:
  new() -> Form with org hierarchy, employee dropdown, locations
    - 150+ hardcoded location options in load_location_options()
  show() -> With eager-loaded status_changes
  pdf() -> Via CriticalInformationPdfGenerator
  download_media() -> Streams attached media file (ActiveStorage)
  edit() / update() -> Standard updates with org hierarchy reload
  create() -> Saves and triggers NotifyTeamsJob if immediate_urgency
  update_status() -> AJAX status update
  approve() -> Sets status to :resolved
  deny() -> Sets status to :cancelled

STRONG PARAMS: employee_id, name, phone, email, agency, division,
               department, unit, incident_type, incident_details, cause,
               impact_started, location, urgency, impact,
               impacted_customers, next_steps, media, staff_involved[]

15.6 GENERATED FORM CONTROLLERS ----------

Follow template pattern from FormGenerator service:
  new() -> Loads org hierarchy, builds prefill_data
  create() -> Sets employee_id, redirects to form_success_path
  pdf() -> Via corresponding PdfGenerator (if approval form)
  approve() / deny() / update_status() -> (if approval form)

Specific controllers include:
  LeaveOfAbsenceFormsController, WorkScheduleOrLocationUpdateFormsController,
  WorkplaceViolenceFormsController, NoticeOfChangeFormsController,
  Osha301FormsController, Rm75FormsController,
  CarpoolFormsController, BrownMailFormsController,
  SocialMediaFormsController, GymLockerFormsController,
  CreativeJobRequestsController

15.7 TASK REASSIGNMENTS CONTROLLER ----------

File: app/controllers/task_reassignments_controller.rb Before action: require_login

ACTIONS:
  reassign() -> Reassigns task to new employee
    - Calls task.reassign_to!(new_assignee_id:, reassigned_by_id:, reason:)
    - Sends TaskReassignmentMailer.reassignment_notification

  take_back() -> Original assignee reclaims task
    - Checks task.can_take_back?(employee_id)
    - Sends TaskReassignmentMailer.take_back_notification

  history() -> AJAX returns reassignment history as JSON

HELPERS:
  find_task() -> Polymorphic task lookup across:
    CriticalInformationReporting, ParkingLotSubmission,
    ProbationTransferRequest, + dynamic form models

==
16. CONTROLLERS - ADMIN & AUTHORIZATION
==

16.1 ACL CONTROLLER ----------

File: app/controllers/acl_controller.rb Before actions: require_system_admin, set_group

CONSTANTS:
  DROPDOWN_ITEMS - 9 permission keys:
    inbox, submissions, reports, dashboards, manage_forms,
    emulate, acl, auth_console, lookup_tables

ACTIONS:
  index() -> Lists all groups with member counts
  show() -> Shows group members, employee search for adding
  new() / create() -> Create new group
  edit() / update() -> Update group name/description
  destroy() -> Deletes group + all permissions + all memberships

  add_member() -> Adds employee to group
  remove_member() -> Removes employee from group

  permissions() -> Shows current dropdown/form permissions for a group
  update_permissions() -> Transaction: destroys and recreates all group
                          permissions (both dropdown and form types)

  org_permissions() -> Shows/manages org-level permissions
                       (agency/division/department/unit hierarchy)
  update_org_permissions() -> Saves org-level permissions

STRONG PARAMS: group_name, description

16.2 AUTHORIZATION CONSOLE CONTROLLER ----------

File: app/controllers/authorization_console_controller.rb Before actions: require_auth_console, set_managed_departments

PURPOSE: Manage authorized approvers for departments/service types. Controls who can approve parking permits, employee IDs, keys, etc.

ACTIONS:
  index() -> Lists approvers filtered by department
    - Auth console admin sees all GSA departments
    - Regular users see only their managed department

new() / create() -> Create authorized approver
- Resolves department from budget units
- Stores authorized_by from session

edit() / update() / destroy() -> Standard CRUD
destroy_all_for_employee() -> Bulk delete all approvals for one employee

HELPERS:
set_managed_departments() -> Determines departments user can manage
fetch_managed_employees() -> Employees in managed departments
fetch_buildings_for_department() -> Hardcoded building names
fetch_managed_budget_units() -> Units with formatted labels
resolve_department_from_units() -> Derives department from first unit

STRONG PARAMS: employee_id, department_id, service_type, key_type,
span, locations[], budget_units[]

16.3 FORM TEMPLATES CONTROLLER ----------

File: app/controllers/form_templates_controller.rb (~1300 lines) Before actions: require_system_admin, set_form_template

PURPOSE: Full form builder interface for creating dynamic forms.

ACTIONS:
index() -> Lists all form templates with metadata (fields, tags, ACL)
new() / create() -> Creates form template with fields, statuses, routing
- Two-pass field creation: first create all fields, then link conditionals
- Generates database table, model, controller, views via FormGenerator

show() -> Display form template details
edit() / update() -> Updates form and all related data
destroy() -> Deletes form template and all related records
duplicate() -> Creates copy of entire form
publish() -> Marks form as active
reorder_fields() -> AJAX drag-and-drop field reordering

HELPERS:
save_routing_steps(), save_statuses(), save_fields()
save_field_conditionals(), link_conditional_references()
generate_form_controller(), generate_form_model()
create_database_table() -> Uses MSSQL-specific SQL

16.4 ADMIN IMPERSONATIONS CONTROLLER ----------

File: app/controllers/admin/impersonations_controller.rb Before action: require_super_admin

ACTIONS:
new() -> Lists employees available for impersonation
create() -> Stores real_admin_id, replaces session with target employee
destroy() -> Restores original admin session

==========
17. CONTROLLERS - REPORTS & DASHBOARDS
==========

17.1 REPORTS CONTROLLER ----------

File: app/controllers/reports_controller.rb Before action: require_system_admin

ACTIONS:
index() -> Shows available forms and user's scheduled reports
generate() -> Routes to one-time or scheduled report creation
status_options() -> AJAX endpoint for form-type status options

HELPERS:
  generate_one_time_report() -> Validates dates, queues ReportGenerationJob
  create_scheduled_report() -> Creates ScheduledReport with next_run
  available_forms() -> Lists all FormTemplates

17.2 DASHBOARDS CONTROLLER ----------

File: app/controllers/dashboards_controller.rb Before action: require_authentication

ACTIONS:
  index() -> Lists forms with dashboards, embeds Power BI config
  embed_token() -> AJAX endpoint to refresh embed token

HELPERS:
  get_embed_config() -> Calls PowerBiService
  powerbi_embed_url() -> Constructs URL from env vars + form config

17.3 SCHEDULED REPORTS CONTROLLER ----------

File: app/controllers/scheduled_reports_controller.rb Before action: require_system_admin

ACTIONS:
  toggle() -> Enables/disables scheduled report
  destroy() -> Deletes scheduled report

==========
18. CONTROLLERS - DATA MANAGEMENT & LOOKUPS
==========

18.1 LOOKUP TABLES CONTROLLER ----------

File: app/controllers/lookup_tables_controller.rb

PURPOSE: Admin interface for GSABSS reference/lookup tables.

CONSTANTS:
  TABLES - 16 static table configurations:
    agencies, divisions, departments, units, activities, functions,
    funds, programs, phases, tasks, objects, revenue_sources,
    major_programs, sub_objects, sub_units, department_funds

ACTIONS:
  index() -> Lists available lookup tables as cards
  show(id) -> Displays table data via raw SQL query
  new(id) -> Form to insert new record
  create(id) -> Inserts via raw SQL (with quoting for safety)

HELPERS:
  set_table_config() -> Finds table config by key
  column_info() -> Raw SQL to get column metadata
  fetch_rows() -> Raw SQL SELECT * FROM table
  connection() -> Gets ActiveRecord connection

18.2 LOOKUPS CONTROLLER (AJAX Dropdowns) ----------

File: app/controllers/lookups_controller.rb

PURPOSE: AJAX endpoints for cascading org hierarchy dropdowns.

ACTIONS:
  agencies()   -> Returns all agencies
  divisions()   -> Returns divisions for agency_id
  departments() -> Returns departments for agency_id + division_id
  units()   -> Returns units for agency_id + division_id + department_id

RESPONSES: Turbo Stream or JSON format

18.3 SAVED SEARCHES CONTROLLER ----------

File: app/controllers/saved_searches_controller.rb

ACTIONS:
  create() -> Saves filter params as SavedSearch for employee
  destroy() -> Deletes saved search

STRONG PARAMS: filter_type, filter_title, filter_category, filter_status,
               filter_date_from, filter_date_to, filter_employee_name,
               filter_employee_id, sort_by, sort_direction

==
19. CONTROLLERS - BILLING & INVOICES
==

19.1 INVOICES CONTROLLER ----------

File: app/controllers/invoices_controller.rb

ACTIONS:
  new() -> Form for invoice generation parameters
  show() -> Generates invoice PDF
    - Calls Tc60Export.call() stored procedure
    - Renders PDF via InvoiceOverlayPdf service
    - Sends as inline PDF response

STRONG PARAMS: sDate, eDate, type, digits, encumbered

19.2 BILLING TOOLS CONTROLLER ----------

File: app/controllers/billing_tools_controller.rb Before action: require_system_admin

ACTIONS:
  new() -> Shows billing operations form
  move_to_production() -> Executes MSSQL stored procedure
  run_monthly_billing() -> Executes MonthlyBilling stored procedure
  backup_staging() -> Executes staging backup procedure
  backup_production() -> Executes production backup procedure

HELPER: run_stored_proc() -> Executes via BillingBase.connection

==
20. CONTROLLERS - API & EXTERNAL INTEGRATIONS
==

20.1 API::DROPDOWNS CONTROLLER ----------

File: app/controllers/api/dropdowns_controller.rb

ACTIONS: divisions, departments, units Returns JSON from MSSQL tables using PascalCase column names.

20.2 API::NHTSA CONTROLLER ----------

File: app/controllers/api/nhtsa_controller.rb

PURPOSE: Proxy for NHTSA VPIC API (CSP blocks direct browser fetch)

ACTIONS:
  makes() -> Fetches vehicle makes for cars
  models() -> Fetches models for a given make/year

HELPER: fetch_nhtsa() -> HTTP wrapper with timeouts and error handling

21.1 FILTERABLE ----------

File: app/controllers/concerns/filterable.rb

PURPOSE: In-memory filtering, sorting, and filter option collection.

METHODS:
  collect_filter_options(collection, field_mappings)
    -> Extracts unique values from collection for filter dropdowns

  apply_filters(collection, filter_configs:, date_filters:)
    -> Filters collection using lambda-based config
    -> Supports text matching, exact matching, date ranges

  sort_collection(collection, sort_by, sort_direction, sort_configs)
    -> Sorts collection using lambda-based sort configurations

  filters_active?(*filter_params)
    -> Returns true if any filter parameter is present

NOTE: Works on in-memory Ruby arrays, not database queries.
      Designed for cross-model collections where SQL isn't possible.

File: config/routes.rb

ROOT: forms#home

AUTHENTICATION:
  GET /auth/callback   -> sessions#create_oauth
  GET /auth/failure   -> sessions#failure
  POST /auth/entra_id   -> sessions#setup
  POST /login   -> sessions#create_legacy (dev/test only; 403 elsewhere)
  DELETE /logout   -> sessions#destroy

ADMIN:
  /admin/impersonations   -> CRUD (new, create, destroy)
  /sidekiq   -> Sidekiq Web UI (mounted)

ACL:
  /acl   -> CRUD + member: add_member, remove_member,
                                  permissions, update_permissions
                                  collection: org_permissions,
                                  update_org_permissions

AUTHORIZATION CONSOLE:
  /authorization_console   -> CRUD + collection: destroy_all_for_employee

BILLING:
  /billing_tools   -> new, create + collection: move_to_production,
                                  run_monthly_billing, backup_staging,
                                  backup_production

INBOX & SUBMISSIONS:
  GET /inboxqueue   -> inbox#queue
  GET /submissions   -> submissions#index
  GET /submissions/status_options -> submissions#status_options
  /saved_searches   -> create, destroy
  /task_reassignments   -> collection: reassign, take_back, history

```
REPORTS:
  GET  /reports                -> reports#index
  POST /reports/generate       -> reports#generate
  GET  /reports/status_options -> reports#status_options
  /scheduled_reports           -> CRUD + member: toggle

DASHBOARDS:
  GET  /dashboards             -> dashboards#index
  POST /dashboards/embed_token -> dashboards#embed_token

FORM TEMPLATES:
  /form_templates              -> Full CRUD

WORKFLOW FORMS (with approve/deny):
  /parking_lot_submissions     -> new, create, index, show + pdf, approve, deny
  /probation_transfer_requests -> new, create, index, show + pdf, approve, deny,
                                  withdraw
  /critical_information_reportings -> new, create, show, edit, update + pdf,
                                      download_media, approve, deny, update_status
  /rm75_forms                  -> CRUD + pdf, approve, deny, update_status
  /osha301_forms               -> CRUD + pdf, approve, deny, update_status
  /leave_of_absence_forms      -> CRUD + pdf, approve, deny, update_status
  /workplace_violence_forms    -> CRUD + pdf, approve, deny, update_status
  /notice_of_change_forms      -> CRUD + pdf, approve, deny, update_status
  /work_schedule_or_location_update_forms -> CRUD + pdf, approve, deny,
                                             update_status

STANDARD FORMS:
  /carpool_forms               -> Full CRUD
  /brown_mail_forms            -> Full CRUD
  /social_media_forms          -> Full CRUD
  /gym_locker_forms            -> Full CRUD
  /creative_job_requests       -> new, create

HELP:
  GET /help                    -> help#index
  /help_tickets                -> new, create, index, show + member: close

LOOKUPS:
  /lookup_tables               -> index, show, new, create
  GET /lookups/agencies        -> lookups#agencies
  GET /lookups/divisions       -> lookups#divisions
  GET /lookups/departments     -> lookups#departments
  GET /lookups/units           -> lookups#units

API:
  GET /api/nhtsa/makes         -> api/nhtsa#makes
  GET /api/nhtsa/models        -> api/nhtsa#models

INVOICING:
  GET /invoice                 -> invoices#show / invoices#new

PWA:
  GET /manifest                -> rails/pwa#manifest
  GET /service-worker          -> rails/pwa#service_worker

DEBUG:
  GET /debug/invoice_grid      -> grid#show
  /reports/matthew_test_report -> show, run
  /reports/matthew_test_yay    -> show, run

MISC:
  GET /forms/home              -> forms#home
  GET /form_success            -> shared#form_success
```

23. VIEWS & LAYOUTS

23.1 MAIN LAYOUT

File: app/views/layouts/application.html.erb

Structure:

```
<header>
  - County of Ventura logo (left, hidden on mobile)
  - Profile dropdown (right): user name, inbox badge count
    - Dropdown: Inbox, Submissions, Reports, Dashboards, Manage Forms,
      Authorization Console, Lookup Tables, Access Control, Emulate,
      Help, Logout
    - Items controlled by dropdown permissions
  - Emulation banner (yellow bar when impersonating)
  - Login button overlay for unauthenticated users
</header>
<div class="app-container">
  <sidebar>
    - Search input (fuzzy matches form names, field labels, tags)
    - Public forms list (legacy hardcoded forms)
    - Dynamic FormTemplate entries
    - Access control filters restricted templates
  </sidebar>
  <main class="content-container">
    <%= yield %>
  </main>
</div>
```

23.2 SHARED PARTIALS

_logo.html.erb -> County of Ventura logo link _sidebar.html.erb -> Form navigation sidebar with search _confirm_modal.html.erb -> Generic deletion confirmation modal (confirm-modal Stimulus controller) _deny_modal.html.erb -> Workflow denial modal with reason textarea (deny-modal Stimulus controller) _reassign_modal.html.erb -> Task reassignment modal with employee selector (reassign-modal Stimulus controller) _readonly_record.html.erb -> Display-only record view as <dl> list form_success.html.erb -> Post-submission success page

23.3 KEY VIEWS

forms/home.html.erb
- Homepage with image slideshow carousel (10s auto-rotation)

inbox/queue.html.erb
- Main approval queue with:
  - 7-column filter form (auto-submit on change)
  - Sortable submission table
  - Row selection -> action panel (task-selection controller)
  - Dynamic action buttons based on FormTemplate config
  - Modals: approve, deny, reassign

submissions/index.html.erb
- User submission tracking with:
  - Multi-filter form (type, title, category, status, date range, employee)
  - Saved search save/load/delete
  - Status timeline display
  - Category-based status badges

reports/index.html.erb
- Form selector, status filter, date range
- One-time vs scheduled toggle
- Frequency/time/format options for schedules

- Existing scheduled reports list

dashboards/index.html.erb
- Form selector dropdown
- Power BI embed container with loading/error states
- Token refresh mechanism

help/index.html.erb
- Tabbed interface (tabs Stimulus controller):
  - Documentation tab: Getting Started, Forms, Inbox, Submissions, Reports, Dashboards, Admin Tools
  - Tickets tab: Support request list and creation

form_templates/index.html.erb
- Template search by name/fields/tags
- Template cards with metadata
- Create/Edit/Delete actions

acl/index.html.erb
- Organization permissions hierarchy
- Group management with member lists
- Permission editing

23.4 TURBO STREAM RESPONSES ----------

lookups/divisions.turbo_stream.erb -> AJAX division dropdown population lookups/departments.turbo_stream.erb -> AJAX department dropdown population lookups/units.turbo_stream.erb -> AJAX unit dropdown population

23.5 MULTI-PAGE FORM VIEWS ----------

Legacy form views (parking_lot_submissions/new, probation_transfer_requests/new, critical_information_reportings/new, etc.) use multi-step page navigation with progress dots. The form_navigation.js module handles page transitions, showing only one page at a time with Previous/Next/Submit buttons.

==========
24. JAVASCRIPT & STIMULUS CONTROLLERS
==========

38 Stimulus controllers registered in app/javascript/controllers/index.js.

24.1 CORE APPLICATION ----------

application.js
- Imports Turbo Rails
- Loads service worker for PWA support
- Initializes all Stimulus controllers
- Turbo load event listener

controllers/application.js
- Initializes Stimulus Application instance
- Debug mode: false

form_navigation.js
- Multi-step form page navigation
- Shows/hides pages based on current step
- Progress dot updates
- Previous/Next/Submit button management

24.2 TASK & WORKFLOW CONTROLLERS ----------

task_selection_controller.js
Targets: row, actionPanel, actionsContainer
Purpose: Row selection in inbox/submissions tables
Methods:
select(event) -> Toggle row selection, show action panel

deselect() -> Clear selection, hide panel
showActions(taskType, taskId) -> Filter and show relevant buttons

approve_modal_controller.js
Targets: backdrop, modal, title, form, select
Values: approvePath, options, title
Purpose: Approval dialog with destination selection (probation)
Methods:
open(event) -> Show modal, populate destination options
close() -> Hide modal

deny_modal_controller.js
Targets: backdrop, modal, form, reason, title
Purpose: Denial reason collection modal
Methods:
open(event) -> Show modal, populate from trigger dataset
close() -> Hide modal, escape key handler

reassign_modal_controller.js
Targets: backdrop, modal, form, title, taskType, taskId, employeeSelect
Purpose: Task reassignment with employee selector
Methods:
open(event) -> Show modal, format task type title
close() -> Hide modal, backdrop click + escape handlers

24.3 NAVIGATION & LAYOUT CONTROLLERS ----------

sidebar_controller.js
Purpose: Mobile hamburger menu and sidebar toggle
Methods: toggle(), close()

profile_dropdown_controller.js
Targets: dropdown
Purpose: User profile menu dropdown in header
Methods:
toggle(event) -> Show/hide dropdown
outsideClick(event) -> Close on outside clicks
Updates aria-expanded for accessibility

slideshow_controller.js
Targets: slide
Purpose: Auto-rotating image carousel on homepage
Methods:
connect() -> Start 10-second interval
showNextSlide() -> Rotate to next slide
disconnect() -> Clear interval

tabs_controller.js
Purpose: Tabbed interface (help page)
Usage: data-tabs-active-tab-value="docs"

24.4 FORM & INPUT CONTROLLERS ----------

auto_submit_controller.js
Purpose: Auto-submit forms on field change
Method: submit() -> requestSubmit() on form element

gsabss_selects_controller.js
Purpose: Cascading org hierarchy selects (Agency -> Division -> Department -> Unit)
Triggers AJAX requests to /lookups/* endpoints on change

phone_controller.js
Purpose: Phone number input formatting (XXX-XXX-XXXX)

choices_controller.js
Purpose: Choices.js enhanced select library integration

conditional_field_controller.js / conditional_fields_controller.js
  Purpose: Show/hide fields based on dropdown conditions

nhtsa_vehicle_controller.js
  Purpose: Vehicle make/model lookup via NHTSA VPIC API proxy

file_preview_controller.js
  Purpose: File upload preview before submission

24.5 SEARCH & FILTER CONTROLLERS ----------

sidebar_search_controller.js
  Targets: input, formLink, formsList
  Purpose: Fuzzy search filtering in sidebar
  Features:
    - Fuzzy matching algorithm with scoring
    - Bonus scoring: form name (+50), tags (+30), fields
    - HTML highlighting for matched characters
    - DOM reordering based on relevance score
    - Shows matched field/tag name as subtitle
    - XSS-safe HTML escaping

status_filters_controller.js
  Targets: type, status, title
  Values: statusOptions, allStatuses, titleOptions, allTitles
  Purpose: Dynamic filter dropdown population on submissions page
  Methods:
    typeChanged() -> Update status/title options for selected form type

saved_search_controller.js
  Targets: dropdown, nameInput, saveForm, deleteForm
  Purpose: Save, load, and delete custom search filters
  Methods:
    load() -> Apply saved search by ID
    toggleSaveForm() -> Show/hide save form
    save(event) -> Create new saved search
    confirmDelete() -> Delete with confirmation

24.6 DASHBOARD CONTROLLERS ----------

dashboards_controller.js
  Targets: formSelect, container, loading, error, errorMessage, reportContainer
  Purpose: Power BI dashboard embedding and token lifecycle
  Methods:
    loadDashboard(event) -> Submit form to reload dashboard
    loadPowerBiSDK() -> Dynamically load Power BI JavaScript SDK
    embedReport(config) -> Embed Power BI report with access token
    scheduleTokenRefresh(expiration) -> Auto-refresh 5 min before expiry
    refreshToken() -> Fetch new access token via AJAX
    retry() -> Retry embedding on failure
    showLoading(), showReport(), showError() -> State management

24.7 ADMIN CONTROLLERS ----------

form_builder_controller.js
  Purpose: Dynamic form template creation modal interface

group_select_controller.js
  Purpose: ACL group selection in admin interface

tag_chips_controller.js
  Purpose: Tag input component (comma-separated tags)

org_permissions_controller.js
  Purpose: Organization permission hierarchy navigation

authorization_select_controller.js
  Purpose: Authorization console selectors

24.8 OTHER CONTROLLERS ------------------------

confirm_controller.js / confirm_modal_controller.js
  Purpose: Generic confirmation dialogs

billing_modal_controller.js / billing_tools_controller.js
  Purpose: Billing-specific modal and interface

reports_controller.js
  Purpose: Report form dynamic options

scheduled_report_form_controller.js
  Purpose: Toggle between one-time and scheduled report forms

urgency_map_controller.js
  Purpose: Urgency field mapping for CIR forms

probation_transfer_request_controller.js
  Purpose: Probation-specific multi-select and validation logic

login_modal_controller.js
  Purpose: Login modal overlay

==
25. STYLESHEETS & DESIGN SYSTEM
==

Architecture: SCSS organized by layer (base, layout, components, pages) Main manifest: app/assets/stylesheets/application.scss

25.1 DESIGN TOKENS (base/_tokens.scss) --

COLORS:
  Primary action: #4caf50 (green)
  Danger/Deny: #e53935 (red)
  Brand navy: #004976 (links, PDF buttons)
  Reassign blue: #3b82f6
  Take-back teal: #10b981
  Text: #333
  Labels: #555
  Borders: #ddd
  Background: #f0f2f5

SHADOWS: 0 6px 18px rgba(0,0,0,0.08) (soft) FONTS: Helvetica Neue, Arial, sans-serif

25.2 LAYOUT STYLES ----------

layout/_layout.scss
  .app-container   -> Flex container (sidebar + content)
  .content-container -> Main content area with responsive padding
  .page-title   -> Centered page headings

layout/_header.scss (246 lines)
  .app-logo-link   -> Positioned logo (hidden on mobile)
  .profile-icon   -> User profile button with inbox badge counter
  .dropdown-content -> Animated profile menu
  .logout-btn   -> Red logout button
  Responsive breakpoints for mobile/tablet

layout/_sidebar.scss (200 lines)
  .sidebar   -> Gradient background, scrollable, 250px width
  .hamburger-btn   -> Mobile menu toggle (hidden on desktop)

.sidebar-overlay -> Mobile backdrop overlay
.nav-link -> Form links with hover effects
.sidebar-search-input -> Search field styling
Mobile: position: fixed with overlay

25.3 COMPONENT STYLES ----------

components/_buttons.scss
.btn -> Base (38px height, 16px padding)
.btn.approve -> Green (#4caf50)
.btn.deny -> Red (#e53935)
.btn.pdf -> Navy (#004976)
.btn.reassign -> Blue (#3b82f6)
.btn.take-back -> Teal (#10b981)
.form-btn -> Alternate button style with variants

components/_tables.scss
.submission-table -> Inbox/submissions data table
.selectable-row -> Row selection highlighting
.sortable-header -> Column headers with sort indicators
.filter-wrapper -> Filter panel container
.filter-form -> Inline filter form
.action-panel -> Selected task action buttons
.status-dropdown -> Inline status change selector

components/_modals.scss
.modal -> Generic modal (login)
.deny-modal-backdrop -> Flex-centered backdrop
.deny-modal -> Container (520px max-width)
.deny-modal__header -> Modal header
.deny-modal__body -> Modal content
.deny-modal__actions -> Modal action buttons

components/_badges.scss
.badge.is-pending -> Gray (#f2f2f2)
.badge.is-in-review -> Yellow (#fef3c7)
.badge.is-approved -> Green (#e8f6ee)
.badge.is-denied -> Red (#fde8e8)
.badge.is-cancelled -> Gray (#f2f2f2)
.badge.is-scheduled -> Indigo (#e0e7ff)

components/_forms.scss
.form-wrapper -> Card container (900px max-width)
.form-group -> Input groups (flex: 1 1 300px)
.form-row -> Multi-column layouts
.progress-dots -> Multi-step indicator dots
.navigation-buttons -> Previous/Next/Submit buttons

components/_slideshow.scss -> Image carousel components/_choices.scss -> Choices.js select styling components/_form_success.scss -> Success page styling

25.4 PAGE-SPECIFIC STYLES ----------

pages/submissions.scss, pages/billing_tools.scss, pages/probation.scss, pages/parking_lot.scss, pages/impersonation.scss, pages/auth_console.scss, pages/cir.scss, pages/form_templates.scss, pages/reports.scss, pages/dashboards.scss, pages/acl.scss, pages/lookup_tables.scss, pages/help.scss

25.5 RESPONSIVE BREAKPOINTS ----------

Mobile (max-width: 768px):
- Hamburger menu shows
- Sidebar becomes fixed overlay
- Profile button becomes circular icon-only
- Logo hidden
- Reduced font sizes and padding

Tablet (769px - 992px):
  - Show initials instead of full name in profile

Desktop (> 992px):
  - Full navigation visible
  - Sidebar always open
  - Full profile button with name

==
26. HELPERS
==

26.1 APPLICATION HELPER (app/helpers/application_helper.rb)
  current_user   -> Session user hash
  inbox_count   -> Count of pending assignments
  format_phone(digits)   -> Format to (XXX) XXX-XXXX
  format_pst(time, format: :short) -> Localize to Pacific Time
  system_admin?   -> Check "system_admins" group
  fetch_acl_groups   -> Group names + IDs from DB

26.2 INBOX HELPER (app/helpers/inbox_helper.rb)
  HARDCODED_FORM_TYPES   -> [CriticalInformationReporting,
                                    ParkingLotSubmission,
                                    ProbationTransferRequest]
  form_template_for(submission)   -> Lookup FormTemplate for class
  dynamic_form?(submission)   -> Is dynamically generated?
  show_inbox_button?(sub, type)   -> Check button visibility
  inbox_pdf_path(submission)   -> PDF route helper
  inbox_edit_path(submission)   -> Edit route helper
  inbox_approve_path(submission)   -> Approve route helper
  inbox_deny_path(submission)   -> Deny route helper
  inbox_update_status_path(sub)   -> Status update route helper
  inbox_status_options(submission) -> Available status enum values

26.3 SUBMISSIONS HELPER (app/helpers/submissions_helper.rb)
  status_badge_class(status)   -> CSS class for status badge
  category_badge_class(category)   -> CSS class from category symbol
  category_label(category)   -> Human-readable label
  normalize_to_category(status)   -> Map legacy status to category

==
27. MAILERS
==

Base: ApplicationMailer (from: "gsa-forms@ventura.org")

27.1 SECURITY MAILER (app/mailers/security_mailer.rb)
  notify(submission)   -> Approved parking permit to
                                    GSA.Security@venturacounty.gov
                                    with PDF attachment
  denied(submission)   -> Denial to submitter with reason
  notify_delegated_approver(sub)   -> Pending approval to delegated approver

27.2 PROBATION MAILER (app/mailers/probation_mailer.rb)
  notify(submission)   -> Approved transfer to
                                    matthew.davoren@ventura.org with PDF
  denied(request)   -> Denial to submitter with reason

27.3 HELP TICKET MAILER (app/mailers/help_ticket_mailer.rb)
  ticket_submitted(subject:, description:, user_name:, user_email:, employee_id:)
                                  -> To matthew.davoren@ventura.org

27.4 TASK REASSIGNMENT MAILER (app/mailers/task_reassignment_mailer.rb)
  reassignment_notification(id)   -> Notifies new assignee
  take_back_notification(id)   -> Notifies when task returned

27.5 REPORT MAILER (app/mailers/report_mailer.rb)
  report_ready(employee, zip_fn, count, form_type, start, end)
                              -> Sends generated report file
  no_submissions_found(employee, form_type, start, end)
                              -> No data notification
  report_generation_failed(employee, form_type, error)
                              -> Error notification

MAIL CONFIGURATION:
  Development SMTP: smtp4.co.ventura.ca.us:25 (no auth, no TLS)
  DevelopmentMailInterceptor: Redirects all mail to ENV['DEV_EMAIL']
                              Preserves original recipients in subject

==
28.   BACKGROUND   JOBS
==

Queue Adapter: Sidekiq Redis: redis://localhost:6379/0 Workers: 3 threads, configurable processes (default 1)

28.1 SCHEDULED JOBS ----------

ProbationRequestsExpiryJob
  Cron: 0 2 * * * (daily at 2am Pacific)
  Purpose: Expire overdue probation transfer requests
  Calls expire_if_due! on each expired record

ScheduledReportsRunnerJob
  Cron: * * * * * (every minute)
  Purpose: Execute due scheduled reports
  Finds ScheduledReport.due, calls execute! on each
  Catches exceptions to continue processing

28.2 TRIGGERED JOBS ----------

NotifySecurityJob
  Trigger: After parking lot submission auto-approval (MB3)
  Sends: SecurityMailer.notify

NotifyProbationJob
  Trigger: After probation transfer request approval
  Sends: ProbationMailer.notify

NotifyTeamsJob
  Trigger: After CIR creation with immediate urgency
  Sends: TeamsNotifier.send_cir_alert (Microsoft Teams webhook)

ReportGenerationJob
  Trigger: One-time report generation or ScheduledReport.execute!
  Supports: CSV and PDF/ZIP formats
  Detects: Date column naming convention dynamically
  Sends: ReportMailer (report_ready, no_submissions_found, or failed)
  Cleanup: Deletes temp files after email delivery

28.3 DEV/TEST JOBS ----------

MatthewTestReportJob -> Uses Sidekiq::Job, calls test service MatthewTestYayJob -> Uses Sidekiq::Job, calls test service

==
29.   RAKE   TASKS
==

29.1 REPORTS (lib/tasks/reports.rake, 520 lines)
  rake reports:new[report_name]   -> Full report scaffolding
  rake reports:destroy[report_name] -> Remove report files

rake reports:run[name,sDate,eDate] -> Execute report synchronously
rake reports:doctor -> Diagnose autoload issues

29.2 POWER BI (lib/tasks/powerbi.rake, 136 lines)
rake powerbi:get_workspace_id -> Fetch workspaces, decode JWT
rake powerbi:test_embed_token -> Test embed token generation

29.3 DEV SEEDS (lib/tasks/dev_seeds.rake)
rake dev:seed:parking -> Seed parking lot data
rake dev:seed:probation -> Seed probation transfer data
rake dev:seed:all -> Seed all test data

29.4 DATA MIGRATION (lib/tasks/migrate_data.rake, 180 lines)
rake db:migrate_from_mssql -> Migrate MSSQL to PostgreSQL
Migrates: Employees, Groups, Employee_Groups, Group_Permissions,
Org hierarchy, Application tables
Remaps: Group IDs, ACL group references

30. CONFIGURATION FILES

config/application.rb
Module: FormsApp
load_defaults: 8.0
Autoload: app/reports
Time zone: Pacific Time (US & Canada)
Queue adapter: :sidekiq
Middleware: Cookies + CookieStore sessions

config/database.yml
Primary: sqlserver (GSABSS env vars)
Billing: Same server, separate pool

config/sidekiq.yml
Queue: default
Threads: 3
Processes: ENV['JOB_CONCURRENCY'] (default 1)
Scheduled: probation_requests_expiry_job (daily 2am)

config/sidekiq_cron_schedule.yml
scheduled_reports_runner (every minute)

config/storage.yml
Development: local disk
Test: tmp/storage
Production options: AWS S3, Google Cloud, Azure Blob (commented)

config/cable.yml
Development: async (in-process)
Production: solid_cable (SQL-backed, 0.1s polling, 1 day retention)

config/importmap.rb
Turbo, Stimulus, all controllers, form_navigation, sortablejs

31. TESTING

Framework: Minitest (Rails default) Parallel: Enabled (test_helper.rb uses available processors) Total test files: 23

Mailer tests: security_mailer, probation_mailer (placeholder) Mailer previews: SecurityMailerPreview, ProbationMailerPreview Job tests: notify_security_job (placeholder) Controller tests: parking_lot, forms, probation_transfer, creative_job,
shared, inbox Model tests: parking_lot_submission, parking_lot_vehicle, agency, division,

department, unit, user, probation_transfer_request,
creative_job_request System tests: probation_transfer_requests

32. DEPLOYMENT & INFRASTRUCTURE

Web Server: Puma (>= 5.0) Deployment: Kamal (Docker-based) Asset Delivery: Thruster (HTTP caching/compression) Boot Optimization: Bootsnap

PWA Support:
GET /manifest -> PWA manifest
GET /service-worker -> Service worker

Environment Variables Required:
GSABSS_HOST, GSABSS_DATABASE, GSABSS_USERNAME, GSABSS_PASSWORD
REDIS_URL (default: redis://localhost:6379/0)
RAILS_MAX_THREADS (default: 5)
JOB_CONCURRENCY (default: 1)
POWERBI_CLIENT_ID, POWERBI_CLIENT_SECRET, POWERBI_TENANT_ID
POWERBI_AUTHORITY_URL, POWERBI_API_URL, POWERBI_SCOPE
APP_HOST (for Teams notification links)
DEV_EMAIL (development mail interceptor)
SMTP_FROM_ADDRESS (report mailer)

33. KEY ARCHITECTURAL PATTERNS

1. SESSION-BASED AUTH: No Devise/Pundit. All auth via session[:user] hash
with manual group membership checks. OmniAuth handles Entra ID OAuth.

2. ORG HIERARCHY CASCADE: Agency -> Division -> Department -> Unit.
Used for: form prefill, permission scoping, approval routing,
dropdown filtering.

3. POLYMORPHIC TRACKING: StatusChange and TaskReassignment use polymorphic
belongs_to for form-agnostic status history and reassignment chains.

4. DUAL FORM SYSTEM: Legacy hardcoded forms (3 main workflow forms) coexist
with dynamically-generated forms via FormTemplate builder. Both appear
in the same inbox and submissions views.

5. IN-MEMORY FILTERING: Filterable concern operates on Ruby arrays, not
database queries. This enables cross-model filtering in inbox/submissions
where data comes from multiple unrelated tables.

6. GENERATED SCAFFOLDING: FormTemplatesController generates complete
model/controller/view/migration files. Uses MSSQL-specific DDL for
table creation.

7. CONCERN-BASED COMPOSITION: TrackableStatus, Reassignable, PhoneNumberable
provide consistent behavior across form models. Models declare which
concerns they include and implement required abstract methods.

8. PERMISSION CASCADING: Org-level permissions cascade from agency down to
unit. Group permissions are merged at runtime. Both types are resolved
in ApplicationController and cached per-request.

9. STATUS CATEGORY NORMALIZATION: Different forms use different status values
but all map to normalized categories (pending, in_review, approved,
denied, cancelled, scheduled) for consistent badge display.

10. MULTI-STEP APPROVAL: Some forms have single-step (submit -> approve),
some have multi-step (submit -> step_1_approved -> step_2_approved),
and some have delegated approval chains.

==
34. DATA FLOW DIAGRAMS
==

34.1 FORM SUBMISSION FLOW ----------

```
Employee                Controller          Model/DB
--------                ----------          --------
Fills multi-page form  -->  new()               Build empty record
Submits              -->  create()              Save record
                          Set employee_id       Record initial status
                          Set status            (TrackableStatus)
                          Route to approver     Determine assignee
                          Queue notification job  (if approval type)
Sees success page     <--  redirect form_success
```

34.2 APPROVAL FLOW ----------

```
Supervisor/Manager      Controller          Model/DB
------------------      ----------          --------
Views inbox          -->  InboxController#queue   Query pending items
Selects submission   -->  (task_selection JS)     Load action panel
Clicks Approve       -->  approve()               Update status
                          Record status change    (TrackableStatus)
                          Queue notification      (Mailer job)
                          Redirect to inbox
```

34.3 REASSIGNMENT FLOW ----------

```
Manager                 Controller          Model/DB
-------                 ----------          --------
Opens reassign modal  -->  (reassign_modal JS)     Show employee list
Selects new assignee  -->  TaskReassignments       Create TaskReassignment
                          #reassign()             Update assignment field
                                                  Send notification email
```

34.4 REPORT GENERATION FLOW ----------

```
Admin                   Controller          Background
-----                   ----------          ----------
Selects form + options -->  ReportsController       ReportGenerationJob
Generates report      -->  #generate()              Queries model for data
                          Queue job                 Generates CSV or PDF/ZIP
                                                    Sends via ReportMailer
                                                    Cleans up temp files
```

34.5 DYNAMIC FORM CREATION FLOW ----------

```
Admin                   Controller          System
-----                   ----------          ------
Designs form template  -->  FormTemplatesController  Save FormTemplate
Adds fields/statuses   -->  #create()                Save FormFields
Saves                -->                             Save FormTemplateStatuses
                          FormGenerator service     Create MSSQL table
                                                    Generate model.rb
                                                    Generate controller.rb
                                                    Generate view.html.erb
                                                    Add route entry
                                                    Add sidebar link
```

==
END OF DOCUMENT
==

This document should be updated as new features are added or existing functionality changes. Key areas

to update:
  - New models -> Sections 9-11
  - New controllers -> Sections 14-20
  - New Stimulus controllers -> Section 24
  - New services -> Section 13
  - New mailers/jobs -> Sections 27-28
  - Route changes -> Section 22
  - Configuration changes -> Section 30